

Plan Modernizacji Hexabid

Jakość Domenowa i Inspiracje Archetypowe

Hexabid Architecture Team

2026-06-27

Table of Contents

Wprowadzenie	1
Kryteria oceny	1
Konwencje zapisu	1
Moduł: hexabid-quantity	2
Ocena bieżąca: BARDZO DOBRA	2
Znalezione problemy	2
Inspiracje archetypowe	2
Zalecane działania	2
Moduł: hexabid-product	3
Ocena bieżąca: DOBRA	3
Znalezione problemy	3
Inspiracje archetypowe	3
Zalecane działania	4
Moduł: hexabid-pricing	5
Ocena bieżąca: BARDZO DOBRA – wzorcowy	5
Znalezione problemy	5
Inspiracje archetypowe	6
Zalecane działania	6
Moduł: hexabid-inventory	7
Ocena bieżąca: ŚREDNIA – po wzbogaceniu (Faza 2)	7
Znalezione problemy	7
Inspiracje archetypowe	7
Zalecane działania	8
Moduł: hexabid-core (Auction, Lot, Party, Domain Events)	9
Ocena bieżąca: DOBRA (Auction) / SŁABA (Party)	9
Znalezione problemy	9
Inspiracje archetypowe	10
Zalecane działania	10
Moduł: hexabid-statements	11
Ocena bieżąca: BARDZO DOBRA – wzorcowy	11
Znalezione problemy	11
Inspiracje archetypowe	12
Zalecane działania	12
Moduł: hexabid-rules (+ hexabid-rules-adapter)	13
Ocena bieżąca: BARDZO DOBRA – wzorcowy	13
Znalezione problemy	13
Inspiracje archetypowe	13
Zalecane działania	14

Moduł: hexabid-auth-core	15
Ocena bieżąca: DOBRA	15
Znalezione problemy	15
Inspiracje archetypowe	15
Zalecane działania	15
Moduł: hexabid-authorization-core	16
Ocena bieżąca: BARDZO DOBRA – wzorcowy	16
Znalezione problemy	16
Inspiracje archetypowe	16
Zalecane działania	16
Moduł: hexabid-payment-core (Accounting)	17
Ocena bieżąca: DOBRA – po przebudowie (Faza 1.1 + 1.2)	17
Znalezione problemy	17
Inspiracje archetypowe	17
Zalecane działania	18
Nowy moduł: hexabid-variance-core (planowany)	19
Inspiracja archetypowa: M10 – Variance (Plan / Execution / Delta)	19
Propozycja	19
Zalecane działania	19
Nowy moduł: hexabid-graph-core (planowane wyodrębnienie)	20
Inspiracja: M05 – Graph	20
Propozycja	20
Zalecane działania	20
Macierz priorytetów	21
Plan fazowy	23
Faza 0: Bugfix [green] ☐ WYKONANE[/]	23
Faza 1: Czystość heksagonalna [green] ☐ CZĘŚCIOWO[/]	23
Faza 2: Wzmocnienie modeli domenowych [green] ☐ CZĘŚCIOWO[/]	23
Faza 3: Zaawansowane inspiracje archetypowe [yellow] — DEFEROWANE[/]	24
Faza 4: Wyodrębnienia i refaktoringi [green] ☐ CZĘŚCIOWO[/]	24
Podsumowanie: Co sprawia, że Hexabid jest (może być) wzorcowy	25
Kluczowe wskaźniki jakości po modernizacji	25

Wprowadzenie

Cel tego dokumentu jest odwrócony względem typowego podejścia "wdróż archetyp X w module Y".

Zaczynamy od **każdego modułu Hexabid z osobna** i oceniamy:

1. **Poprawność domenowa** – czy model chroni niezmienniki, czy logika biznesowa jest tam, gdzie powinna, czy występują bugi.
2. **Czystość heksagonalna** – czy moduł domenowy nie zależy od Spring/JPA, czy granice portów i adapterów są zachowane.
3. **Bogactwo modelu** – czy obiekty domenowe mają zachowania, czy są anemiczne.
4. **Wartość edukacyjna** – czego może nauczyć się programista czytający ten kod.

Dokumentacja kursu [Archetypy Oprogramowania](#) (M01–M10) służy jako **katalog inspiracji** – nie jako lista wymagań. Każda inspiracja jest opatrzona adnotacją, czy i jak podnosi jakość architektury.

Kryteria oceny

Poziom	Etykieta	Opis
P1	BLOKER	Bug domenowy – system może policzyć/zaksięgować błędną wartość
P2	DESIGN	Naruszenie czystości heksagonalnej, logika domenowa w adapterze, anemiczny model
P3	ROZSZERZENIE	Brak funkcji, która wg archetypu podniosłaby jakość/edukacyjność, ale nie powoduje błędów
P4	INSPIRACJA	Koncept z kursu AO wart rozważenia przy okazji większych zmian

Konwencje zapisu

- **[archetyp: M02]** – odniesienie do modułu kursu Archetypy Oprogramowania
- **[BUG]** – krytyczny problem wymagający natychmiastowej naprawy
- **[hexagona1]** – naruszenie architektury heksagonalnej
- **[edu]** – walor edukacyjny wart podkreślenia/zachowania

Moduł: hexabid-quantity

Ocena bieżąca: BARDZO DOBRA

`Quantity(amount, unit)` i `Unit(symbol, name)` to wzorcowe value objecty: immutable (record), chronią niezmienniki (`Objects.requireNonNull`, brak ujemnych wartości), mają własny `compareTo` z walidacją kompatybilności jednostek.

`Unit` jako record (nie enum) pozwala na rozszerzanie bez modyfikacji – celowa decyzja projektowa pokazująca Open/Closed Principle w praktyce.

Znalezione problemy

Priorytet	Gdzie	Problem
~~P1 BLOKER~~ NAPRAWIONY	<code>Quantity.subtract()</code> (linia 37)	Dodano guard z <code>requireArgument</code> i opisowym komunikatem. Test w <code>QuantityTest</code> .
P4 INSPIRACJA	<code>hexabid-quantity</code>	Brak <code>Quantity.multiply(BigDecimal) / divide(BigDecimal)</code> . Przydatne przy wycenie per-unit – ale to już pokrywa <code>hexabid-pricing</code> .

Inspiracje archetypowe

- `[M02]` Product tracking strategy używa `Quantity` w kontekście identyfikowalności – warto pokazać w dokumentacji, jak `Quantity` łączy product i inventory.
- `[M08]` Inventory może operować na `QuantityRange(min, max)` zamiast gołego `Quantity` – to rozszerzenie warto rozważyć przy przebudowie inventory.

Zalecane działania

1. Dodać guard w `subtract()` – najwyższy priorytet.
2. Dodać `Quantity.ZERO` jako stałą.
3. Dodać `multiply(BigDecimal)` i `divide(BigDecimal)` – niski koszt, wysoka przydatność edukacyjna.

Moduł: hexabid-product

Ocena bieżąca: DOBRA

Moduł rozróżnia definicję produktu (**ProductType**) od oferty komercyjnej (**CatalogEntry**). Sealed interface **Product** z **ProductType** (leaf) i **PackageType** (composite) to poprawna implementacja wzorca Composite.

ProductTrackingStrategy (UNIQUE / INDIVIDUALLY_TRACKED / BATCH_TRACKED / IDENTICAL) to rzadko spotykana w projektach dbałość o warianty śledzenia produktów – doskonały materiał do nauki modelowania domenowego.

Znalezione problemy

Priorytet	Gdzie	Problem
P2 DESIGN	ProductFacade	Łączy rolę repozytorium, serwisu domenowego i fasady aplikacyjnej. W heksagonalnej architekturze powinien być rozdzielony na: ProductCatalog (port in) + adapter in-memory.
P2 DESIGN	CatalogEntry	Brak jawnego Applicability – przydatność oferty jest albo Validity (zakres dat) albo nie ma jej wcale. Kurs M02 sugeruje osobny obiekt Applicability dla warunków kontekstowych.
P3 ROZSZERZENIE	Produkt	Brak Variance jako first-class obiektu. Warianty produktu są modelowane przez ProductFeatureType + ProductFeatureInstance , ale nie ma formalnej relacji "wariant bazuje na produkcie bazowym".

Inspiracje archetypowe

- **[M02] ProductDefinition / CatalogEntry / ProductInstance** – kurs wyraźnie rozdziela trzy światy. Hexabid ma **ProductType** (definicja) i **CatalogEntry** (katalog), ale **ProductInstance** jest w **hexabid-inventory**. To **poprawne** rozdzielenie modułowe, warte podkreślenia w dokumentacji.
- **[M02 L06] Kremenaros** – model trzech granic (definicja / sprzedaż / realizacja) jest w Hexabid zrealizowany przez trzy osobne moduły. To wzorcowe.

Zalecane działania

1. Rozdzielić `ProductFacade` na `ProductCatalog` (port in) + adapter in-memory.
2. Dodać `Variant` jako value object łączący `ProductType` z zestawem cech (`Map<ProductFeatureType, ProductFeatureValue>`).
3. Przenieść mapowanie `CatalogEntry.validity` do osobnego obiektu `Validity` (obecnie inline).

Moduł: hexabid-pricing

Ocena bieżąca: BARDZO DOBRA – wzorcowy

To najlepiej zaprojektowany moduł domenowy w projekcie. Separacja `PriceCalculator` (matematyka) od `PricingComponent` (semantyka) i `CompositePriceComponent` (agregacja) to tekstbookowa implementacja wzorca Strategy + Composite. `Money` to pełnoprawny value object z arytmetyką.

Trzy warstwy ceny (Calculator / Component / Composite) można pokazać studentom jako praktyczny przykład, jak uniknąć anemicznego `price` pola. Każda warstwa ma inną odpowiedzialność i inną częstotliwość zmian – zgodnie z [M04 L02] three zones of change.

Znalezione problemy

Priorytet	Gdzie	Problem
P1 BLOKER CZĘŚCIOWO	<code>AuctionPricingFacade.calculate()</code> (linia 31)	Excise naliczane od netto (hammer - wadium), powinno być od pełnego hammer price. Podatek akcyzowy dotyczy całej transakcji, a nie kwoty pomniejszonej o wadium. Nadal otwarte.
P1 BLOKER NAPRAWIONY	<code>createExciseCalculator()</code> <code>ExciseType</code> (linia 34)	<code>createExciseCalculator()</code> zawsze używał <code>PercentageCalculator</code> nawet dla <code>ExciseType.PER_UNIT</code> . Naprawiono: switch na <code>ExciseType</code> — <code>PercentageCalculator</code> dla <code>PERCENTAGE</code> , <code>PerUnitCalculator</code> dla <code>PER_UNIT</code> . Dodano <code>withFixedParam()</code> w Builderze dla przekazywania <code>quantity</code> .
P2 DESIGN	<code>AuctionPricingFacade</code>	If-else chain dla <code>isExcisable()</code> / <code>isImported()</code> to logika warunków zastosowania (applicability) rozsmarowana w kodzie proceduralnym. Powinna być modelem – patrz inspiracje poniżej.
P3 ROZSZERZENIE	<code>VatRate</code> , <code>ExciseRate</code> , <code>CustomsDutyRate</code>	Brak <code>validFrom</code> / <code>validTo</code> – stawki nie są wersjonowane w czasie.

Inspiracje archetypowe

- [M03 L03] **Calculator / Component / Interpretation** – Hexabid ma Calculator i Component, ale brak jawnego **Interpretation** enum (TOTAL/UNIT/MARGINAL). Dodanie go pozwoliłoby usunąć if-y na typie kalkulatora.
- [M03 L04] **Applicability** – warunki zastosowania komponentu pricingowego powinny być modelowane jako `PricingComponent.applicability()` zwracające `ApplicabilityRule`, a nie if-else w fasadzie.
- [M03 L05] **Validity** – wersjonowanie stawek w czasie to naturalne rozszerzenie zwiększające realizm produkcyjny.

Zalecane działania

1. **Natychmiast:** naprawić bug Excise (P1).
2. **Natychmiast:** naprawić dead code w `ExciseRate.applyTo()` (P1).
3. Dodać `Validity(Instant validFrom, Instant validTo)` do stawek (P3).
4. Dodać **Interpretation** enum i powiązać z kalkulatorami (P3).
5. Dodać `ApplicabilityCondition` jako first-class obiekt przy `PricingComponent` (P3).
6. Dodać `PricingCatalog` port do lookupu stawek (P3).

Moduł: hexabid-inventory

Ocena bieżąca: ŚREDNIA – po wzbogaceniu (Faza 2)

`InventoryEntry` i `Instance` stanowią core, ale moduł nie realizuje podstawowego zadania inwentarza: zarządzania dostępnością. Nie ma rezerwacji, alokacji, stanów magazynowych ani ruchów inwentarzowych.

Moduł można pokazać jako **antywzorzec modelowania dosłownego** ([M08 L02]) – zamiast abstrakcyjnego "Inventory = ograniczony zasób w czasie", mamy dosłowne "lista instancji w mapie". To celowe niedomodelowanie, które uwidacznia się dopiero przy próbie rezerwacji towaru po wygranej aukcji.

Znalezione problemy

Priorytet	Gdzie	Problem
P2 DESIGN	<code>InventoryFacade</code>	Analogicznie do <code>ProductFacade</code> – łączy repozytorium i serwis domenowy. Brak wyraźnego portu <code>InventoryAvailability</code> .
P2 DESIGN NAPRAWIONY	<code>InventoryEntry.addInstance()</code>	Mimo <code>Set<InstanceId></code> , <code>instanceDetails</code> to <code>Map<InstanceId, Instance></code> – dodanie istniejącego ID cicho nadpisze wpis. Dodano guard na duplikat.
P3 ROZSZERZENIE NAPRAWIONE	<code>hexabid-inventory</code>	Brak modelu dostępności. Dodano <code>Availability</code> , <code>Reservation</code> aggregate, <code>ReservationService</code> (reserve/confirm/cancel/complete), <code>InventoryMovement</code> .
P3 ROZSZERZENIE	<code>hexabid-inventory</code>	Brak zarządzania ilościowego dla produktów IDENTICAL.
P2 DESIGN	<code>InventoryFacade</code>	Analogicznie do <code>ProductFacade</code> – łączy repozytorium i serwis domenowy. Brak wyraźnego portu <code>InventoryAvailability</code> .

Inspiracje archetypowe

- [M08 L03] **Availability = total - allocated** – fundament, który w Hexabid nie istnieje. Należy dodać `Availability(ProductIdentifier, Quantity total, Quantity allocated)` z metodą `available()` → `Quantity`.

- [M08 L04] **Reservation** – tymczasowa blokada zasobu na czas trwania aukcji. Powinna wygasać (TTL) i być powiązana z **AuctionId**.
- [M08 L05] **Allocation** – trwałe przypisanie zasobu do zamówienia po wygranej.
- [M08] Inventory to nie magazyn – to **dowolny ograniczony zasób**: czas lekarza, miejsce w paczkomacie, pula kredytowa. Hexabid powinien modelować Inventory właśnie tak abstrakcyjnie, a nie jako "lista rzeczy w magazynie".

Zalecane działania

1. Dodać **Availability** z **total()**, **allocated()**, **available()** (P3).
2. Dodać **Reservation(reservedBy, reservedAt, expiresAt, quantity)** (P3).
3. Dodać **Allocation(allocatedTo, allocatedAt, quantity)** (P3).
4. Dodać **InventoryMovement(movementType, quantity, reason, timestamp)** (P3).
5. Zabezpieczyć **addInstance()** przed duplikatami (P2).
6. Rozdzielić **InventoryFacade** na port + adapter (P2).

Moduł: hexabid-core (Auction, Lot, Party, Domain Events)

Ocena bieżąca: DOBRA (Auction) / SŁABA (Party)

Auction aggregate to bogaty model ze stanową maszyną (8 statusów), chronionymi tranzycjami (`ensureStatusIs`, `ensureInProgressAt`), zdarzeniami domenowymi (10 typów) i czystą separacją use case'ów od logiki agregatu.

Natomiast `Party` to anemiczny rekord (tylko `partyId` + `displayName` + `roles`), a `Auction.edit()` ma podejrzaną implementację niszczącą oryginalny lot.

Auction aggregate to wzorcowy przykład **bogatego modelu domenowego** z DDD: każda metoda chroni niezmiennik stanowy, a use case'y delegują logikę do agregatu. `PlaceBidService` pokazuje, jak łączyć KYC, reguły i domenę bez rozlewania logiki.

Znalezione problemy

Priorytet	Gdzie	Problem
P2 DESIGN NAPRAWIONY	<code>Auction.edit()</code> (linia 105)	Tworzył nowy <code>Lot.singleProductDraft()</code> , niszcząc referencję. Dodano <code>EditAuctionService</code> z <code>EditAuctionUseCase</code> result types: <code>AuctionEdited</code> , <code>AuctionNotFound</code> , <code>EditNotAllowed</code> (guard na DRAFT/PUBLISHED).
P2 DESIGN NAPRAWIONY	<code>Auction.assignParticipationPolicyTemplate()</code>	Setter bez guarda. Naprawiono przez <code>EditAuctionService</code> – dozwolone tylko w DRAFT i PUBLISHED.
P2 DESIGN	<code>Party</code> (rekord)	Anemiczny – tylko dane, zero zachowań. Brak <code>hasRole(PartyRole)</code> , <code>isBuyer()</code> , <code>isSeller()</code> , <code>isActive()</code> , <code>displayName()</code> formatter.
P3 ROZSZERZENIE	<code>hexabid-core</code>	Brak <code>Order</code> aggregate po wygranej aukcji. Winner zostaje wyłoniony, ale nie ma obiektu reprezentującego "zakup" łączącego aukcję z płatnością i wysyłką.

Priorytet	Gdzie	Problem
P3 ROZSZERZENIE	hexabid-core	Bid jako value object jest OK, ale w momencie gdy trzeba obsłużyć retrakcję (wycofanie oferty), brak własnego ID boli – nie ma jak odwołać się do konkretnej oferty.

Inspiracje archetypowe

- [M07] **Party** – Kurs AO pokazuje, że "user to za mało". **Party** powinno wspierać: **PartyType** (PERSON/ORGANIZATION), hierarchię (**parentPartyId**), relacje (**PartyRelationship**), kontakt (**email**, **phone**). To naturalne rozszerzenie dla aukcji z udziałem firm i pełnomocników.
- [M09] **Ordering** – **Order** jako osobny aggregate pojawia się po wygranej aukcji i łączy **AuctionId** + **BuyerId** (PartyId) + **LotId** + **Price** (Money) **PaymentDeadline**. To pomost pomiędzy **hexabid-core** a **hexabid-payment-core**.
- [M09 L02] Order jako **wielokrotnego użytku szkielet pojęciowy** – w Hexabid rolę ordera pełni Auction, ale tylko do momentu zamknięcia. Po wygranej potrzebny jest osobny byt.

Zalecane działania

1. □ Naprawić **Auction.edit()** – implementacja EditAuctionService (P2).
2. □ Dodać guard stanu w **assignParticipationPolicyTemplate()** (P2).
3. Wzbogacić **Party** o metody **hasRole()**, **isBuyer()**, **isSeller()** (P2) – deferowane.
4. Dodać **Order** aggregate – most pomiędzy auction a payment (P3) – deferowane.
5. Dodać zdarzenia domenowe **AuctionWonEvent** □ tworzy **Order** (P3) – deferowane.

Moduł: hexabid-statements

Ocena bieżąca: BARDZO DOBRA – wzorcowy

`StatementDependencyGraph` to pełna implementacja DAG-a z DFS reachability, sortowaniem topologicznym, detekcją cykli i budowniczym. `ParticipationPolicyEvaluator` poprawnie łączy template + graf + decyzję. `PolicyTemplateCatalog` z 3 pre-built template'ami pokazuje Factory pattern w praktyce.

To najlepszy przykład w projekcie, że **graf to nie tylko algorytmika** ([M05 L01]). Problem biznesowy (kolekcja oświadczeń kandydata) jest wprost modelowany jako DAG, a nie jako lista kroków w serwisie. Algorytmy grafowe rozwiązują realny problem biznesowy (kaskadowe odrzucenie zależnych oświadczeń).

Znalezione problemy

Priorytet	Gdzie	Problem
P2 DESIGN	<code>StatementService.submitAnswer()</code>	Kaskada odrzucenia liczona w 2 miejscach: w serwisie (linia 141-143) i w <code>ParticipationPolicyEvaluator</code> (linia 63). Duplikacja logiki.
P2 DESIGN NAPRAWIONY	<code>hexabid-statements</code>	Dodano <code>StatementDomainEvent</code> sealed interface z 4 eventami: <code>ProgramStarted</code> , <code>AnswerSubmitted</code> , <code>ProgramCompleted</code> , <code>ProgramRejected</code> . <code>StatementEventPublisher</code> output port. Zintegrowane w <code>StatementService</code> .
P3 ROZSZERZENIE	<code>hexabid-statements</code>	Brak <code>AuctionKind</code> jako sealed interface. ADR zakładał <code>EnglishAscending</code> / <code>SealedBidTender</code> / <code>RestrictedTender</code> / <code>DutchDescending</code> , ale kod ma tylko string <code>participationPolicyTemplate</code> w <code>Auction</code> .
P3 ROZSZERZENIE	<code>hexabid-statements</code>	Brak <code>PhaseTransitionGuard</code> – [M04] <code>RuleAlgebra</code> mogłoby służyć do gatingu przejść między fazami (IDENTITY $\square$ QUALIFICATION $\square$ CAPABILITY $\square$ COMMITMENT).

Inspiracje archetypowe

- [M05] **Graph jako archetyp myślowy** – `StatementDependencyGraph` to gotowy materiał na osobny moduł `hexabid-graph-core` z generycznym DAG + algorytmami, z którego korzysta `hexabid-statements`. To podniosłoby reużywalność.
- [M04 L02] **Three zones of change** – w `statements` widać to naturalnie: stabilne (`StatementDependencyGraph`, algorytmy), domknięcia (`ParticipationPolicyEvaluator`), konfiguracja (`PolicyTemplateCatalog`). Warto to wyeksponować w dokumentacji.
- [M04] **RuleAlgebra** – `hexabid-rules` ma Boolean/Compliance/Explained algebra. `Statements` mógłby użyć `ComplianceAlgebra` do phase gating zamiast if-else na statusach.

Zalecane działania

1. □ Dodać zdarzenia domenowe fazy (P2).
2. Wyeliminować duplikację kaskady (P2) – deferowane.
3. Dodać zdarzenia domenowe do `StatementProgramInstance` (P2).
4. Dodać `AuctionKind` sealed interface i zintegrować z `Auction` (P3).
5. Rozważyć wyodrębnienie `hexabid-graph-core` (P4).

Moduł: hexabid-rules (+ hexabid-rules-adapter)

Ocena bieżąca: BARDZO DOBRA – wzorcowy

`Expression` sealed interface z 7 typami + `ExpressionVisitor` + `RuleAlgebra<R>` to najczystsza implementacja wzorca Interpreter + Visitor w projekcie. Trzy implementacje algebry (Boolean, Compliance, Explained) pokazują, jak jeden AST może być ewaluowany na różne sposoby w zależności od potrzeby.

To moduł, który powinien obowiązkowo obejrzeć każdy programista uczący się wzorców projektowych. Pokazuje: * **Interpreter pattern** – AST dla reguł biznesowych * **Visitor pattern** – separacja struktury od operacji * **Algebra** – abstrakcja semantyki ewaluacji * **Three zones of change** ([M04 L02]) – stabilny AST, domknięcia (algebry), selekcja (RuleCatalog + RuleDefinition)

Znalezione problemy

Priorytet	Gdzie	Problem
P3 ROZSZERZENIE	<code>hexabid-rules</code>	Brak <code>RuleVersion</code> – reguły nie mają wersjonowania, nie można śledzić zmian w definicji reguły w czasie.
P3 ROZSZERZENIE	<code>hexabid-rules</code>	Brak persistencji – <code>RuleCatalog</code> to <code>CopyOnWriteArrayList</code> w pamięci. Dla produkcyjnego użycia reguły muszą być ładowane z bazy/serwera.
P3 ROZSZERZENIE	<code>hexabid-rules-adapter</code>	Moduł nie jest podpięty do <code>hexabid-pricing</code> (applicability pricing rules) ani do <code>hexabid-statements</code> (phase gating). Potencjał cross-modułowy niewykorzystany.

Inspiracje archetypowe

- [M04 L05] **Dynamiczne reguły** – biznes zmienia reguły bez deploy'u. W Hexabid to byłoby możliwe przez serializację `Expression` AST do JSON/YAML i hot-reload w RuleCatalog.
- [M04 L06] **Reguły jako osobna bounded context** – `hexabid-rules` już jest osobnym modułem. Warto pokazać, że nie ma zależności od Spring/JPA, co czyni go idealnym kandydatem do wyodrębnienia jako microserwis.

Zalecane działania

1. Dodać `RuleVersion(validFrom, validTo, versionNumber)` (P3).
2. Dodać serializację `Expression` do JSON (P3).
3. Podpiąć `RuleAlgebra` do applicability w pricing (P3).
4. Podpiąć `RuleAlgebra` do phase gating w statements (P3).

Moduł: hexabid-auth-core

Ocena bieżąca: DOBRA

Moduł czysty – **nie znaleziono zależności od Springa** (wbrew ostrzeżeniu w AGENTS.md, które prawdopodobnie dotyczy starszej wersji kodu). `AuthenticatedUser` to poprawny rekord DTO dla principala. `CurrentUserProvider` jako port out jest właściwą abstrakcją.

`CurrentUserProvider` jako **port** (interfejs w domenie) z **adapterami** (`RestCurrentUserProvider` w REST, `WebSocketCurrentUserProvider` w WS) to tekstbookowy przykład Ports & Adapters w warstwie autoryzacji.

Znalezione problemy

Priorytet	Gdzie	Problem
P2 DESIGN	hexabid-auth-core	Zależność <code>spring-boot-starter-security</code> w <code>pom.xml</code> – choć kod nie importuje Springa, sama zależność w domyślnym classpath może ciągnąć niechciane transitive dependencies. Należy usunąć lub oznaczyć <code><scope>provided</scope></code> .
P3 ROZSZERZENIE	hexabid-auth-core	Brak modelu weryfikacji tożsamości – <code>UserVerificationStatusPort</code> istnieje, ale nie ma enum <code>VerificationLevel</code> (NONE / BASIC / ADVANCED / BIOMETRIC).

Inspiracje archetypowe

- [M07] Tożsamość i rola – `AuthenticatedUser` jest tymczasowym kontekstem sesji. Stała tożsamość użytkownika powinna być w `Party`. Hexabid to rozdziela (auth-core vs core.Party) – poprawnie.

Zalecane działania

1. Oznaczyć `spring-boot-starter-security` jako `provided` lub usunąć (P2).
2. Dodać `VerificationLevel` enum do portu weryfikacji (P3).

Moduł: hexabid-authorization-core

Ocena bieżąca: BARDZO DOBRA – wzorcowy

`Permission(ResourceType, Action, Relation)` to elegancki model 3D unikający eksplozji płaskich nazw. `OrganisationCode` z `isSameOrBelow()` do hierarchicznej kontroli dostępu. `AuthorizationContextFactory` jako czysta Java z portem `SubordinateQueryPort`.

Model 3D uprawnień to rzadko spotykana w projektach dojrzałość. Większość systemów ma `Role.ADMIN` / `Role.USER` (płaskie role). Hexabid pokazuje, że przestrzeń uprawnień to iloczyn kartezjański: **co?** (`ResourceType`) × **jaka operacja?** (`Action`) × **czyj zasób?** (`Relation`). To daje `Permission(AUCTION, EDIT, OWN)` zamiast `AUCTION_EDIT_OWN` jako string.

Znalezione problemy

Priorytet	Gdzie	Problem
P3 ROZSZERZENIE	<code>ResourceType</code>	Tylko <code>AUCTION</code> i <code>REPORT</code> – zbyt mało dla pełnego systemu. Brak <code>PRODUCT</code> , <code>INVENTORY</code> , <code>PAYMENT</code> , <code>PARTY</code> , <code>STATEMENT</code> , <code>SETTINGS</code> .
P3 ROZSZERZENIE	<code>hexabid-authorization-core</code>	<code>AuthorizationContext</code> sprawdza czy użytkownik ma prawo edytować aukcje , ale nie czy użytkownik może edytować tę konkretną aukcję . Resource-level enforcement jest w use case'ach (ręcznie w <code>ActivateAuctionService</code> linia 34) – powinien być w <code>AuthorizationContext</code> .

Inspiracje archetypowe

- **[M07] Party + Scope** – `OrganisationCode` to praktyczna implementacja hierarchicznego scope'u. Warto pokazać w dokumentacji, że to wariant wzorca "scope" z **[M07] Party Relationship**.

Zalecane działania

1. Rozszerzyć `ResourceType` o brakujące zasoby (P3).
2. Dodać resource-level checking do `AuthorizationContext` (P3).

Moduł: hexabid-payment-core (Accounting)

Ocena bieżąca: DOBRA – po przebudowie (Faza 1.1 + 1.2)

Moduł ma poprawną strukturę (AccountingEntry z DEBIT/CREDIT, AccountingTransaction z listą entries), ale implementacja jest sketchem, nie produkcyjnym modelem księgowym.

Moduł można pokazać jako **etap pośredni modelowania** – widać, że autor zna koncepcję double-entry, ale nie dokończył implementacji. To materiał na case study: "jak rozpoznać niedomodelowany accounting".

Znalezione problemy

Priorytet	Gdzie	Problem
P1 BLOKER NAPRAWIONY	<code>AccountingTransaction.isBalanced()</code> (linia 21-23)	Zawsze zwracało <code>true</code> przez <code>var</code> typowanie. Naprawiono: poprawna suma DEBIT i CREDIT, konstruktor wymaga 2 entry, dodano <code>createBalanced()</code> .
P1 BLOKER NAPRAWIONY	<code>ProcessPaymentUseCase.execute()</code> (linia 49-52)	Tworzył jeden DEBIT. Naprawiono: DEBIT (buyer) + CREDIT (seller).
P2 DESIGN NAPRAWIONY	hexabid-payment-core	Spring usunięty z domeny: <code>PaymentGatewayRegistry</code> usunięty, <code>AuctionWonEventListener</code> przeniesiony do hexabid-bootstrap jako <code>PaymentEventSubscriber</code> . <code>spring-context</code> usunięty z pom.xml.
P3 ROZSZERZENIE NAPRAWIONE	hexabid-payment-core	Dodano <code>AccountType</code> (ASSET/LIABILITY/EQUITY/REVENUE/EXPENSE), <code>ChartOfAccounts</code> , <code>Ledger</code> (post/balance), <code>Account</code> z typem konta.
P3 ROZSZERZENIE NAPRAWIONE	hexabid-payment-core	Dodano agregat <code>Payment</code> z cyklem życia (PENDING→COMPLETED/FAILED→REFUNDED) i <code>PaymentId</code> . Używa <code>Money</code> z hexabid-pricing.

Inspiracje archetypowe

- [M06 L02] Rzeczywista struktura wartości – kwota to nie pole `amount`. Accounting powinien

mieć: komponenty, semantykę, kontekst. `AccountingEntry.type` (DEBIT/CREDIT) to dopiero początek.

- [M06 L04] **Double-entry ledger** – każda transakcja musi mieć co najmniej dwa wpisy. `isBalanced()` musi faktycznie sprawdzać: `sum(debits) == sum(credits)`.
- [M06 L06] **Audit trail** – append-only log wpisów księgowych. Hexabid powinien mieć zasadę: "entries are never updated, only appended".

Zalecane działania

1. **Natychmiast:** naprawić `isBalanced()` (P1).
2. **Natychmiast:** dodać drugi entry (CREDIT) w `ProcessPaymentUseCase` (P1).
3. Przenieść `PaymentGatewayRegistry` i `AuctionWonEventListener` do adaptera (P2).
4. Dodać `AccountType`, `ChartOfAccounts`, `Ledger` (P3).
5. Zmigrować z `Price` na `Money` (P3).
6. Dodać `Payment` aggregate (INITIATED → PENDING → CONFIRMED → REFUNDED) (P3).

Nowy moduł: hexabid-variance-core (planowany)

Inspiracja archetypowa: M10 – Variance (Plan / Execution / Delta)

Kurs AO pokazuje, że w każdym biznesie istnieją dwa światy: **plan** (co miało się wydarzyć) i **execution** (co się wydarzyło), a kluczową wartością biznesową jest **delta** – różnica między nimi.

W Hexabid ten archetyp nie jest obecnie zaimplementowany w ogólnej formie. Konkretnie przypadki (np. planned hammer price vs actual hammer price, planned close date vs actual close date) są rozsmarowane w logice use case'ów.

Propozycja

Nowy moduł **hexabid-variance-core** z generycznym modelem:

```
Plan<T>(T target, Instant plannedAt)
Execution<T>(T actual, Instant executedAt)
Delta<T>(Plan<T> plan, Execution<T> execution, VarianceComputation<T>
computation)
```

gdzie **VarianceComputation<T>** to interfejs z metodą **compute(Plan<T>, Execution<T>)** → **VarianceResult**.

Zalecane działania

1. Stworzyć **hexabid-variance-core** jako pure Java module.
2. Zaimplementować generyczny **Plan**, **Execution**, **Delta** (P3).
3. Zastosować w kontekście aukcji: **AuctionPriceVariance**, **AuctionScheduleVariance** (P3).
4. Dodać testy pokazujące różne typy wariacji (kwotowa, czasowa, stanowa) (P3).

Nowy moduł: hexabid-graph-core (planowane wyodrębnienie)

Inspiracja: M05 – Graph

`StatementDependencyGraph` w `hexabid-statements` to pełna implementacja DAG-a. Warto wyodrębnić generyczne algorytmy grafowe do osobnego modułu, z którego korzystają `statements` i potencjalnie inne moduły.

Propozycja

```
Graph<Node, Edge>(Set<Node>, Set<Edge>)  
Dag<Node, Edge> extends Graph (z detekcją cyklu)  
TopologicalSort<Node>  
Reachability<Node>  
ConnectedComponents<Node>
```

Zalecane działania

1. Wyodrębnić `hexabid-graph-core` z `hexabid-statements` (P4).
2. Zrefaktoriować `StatementDependencyGraph` aby używał generycznego DAG-a.
3. Dodać testy dla każdego algorytmu.

Macierz priorytetów

Priorytet	Moduł	Problem	Typ
P1	hexabid-pricing	Excise naliczany od netto zamiast hammer price	BUG
P1	hexabid-pricing	ExciseRate.applyTo() PER_UNIT dead code	BUG
P1	hexabid-payment-core	isBalanced() zawsze true	BUG
P1	hexabid-payment-core	Tylko 1 entry (DEBIT) zamiast 2 (DEBIT+CREDIT)	BUG
P1	hexabid-quantity	subtract() może dać ujemny Quantity	BUG
P2	hexabid-payment-core	Spring @Component/@EventListener w core	hexagonal
P2	hexabid-core	Auction.edit() niszczy lot	DESIGN
P2	hexabid-core	Party anemiczne	DESIGN
P2	hexabid-core	assignParticipationPolicyTemplate bez guarda	DESIGN
P2	hexabid-adapter-in-rest	Wadium w ConcurrentHashMap (logika domenowa w adapterze)	hexagonal
P2	hexabid-statements	Brak eventów domenowych	DESIGN
P2	hexabid-statements	Duplikacja kaskady w 2 miejscach	DESIGN
P2	hexabid-inventory	addInstance() brak guarda na duplikat	DESIGN
P2	hexabid-product	ProductFacade łączy repo + service	hexagonal
P2	hexabid-auth-core	spring-boot-starter-security w pom.xml	hexagonal
P3	hexabid-inventory	Brak Availability/Reservation/Allocation	ROZSZERZENIE
P3	hexabid-core	Brak Order aggregate	ROZSZERZENIE
P3	hexabid-pricing	Brak Validity na stawkach	ROZSZERZENIE

Priorytet	Moduł	Problem	Typ
P3	hexabid-pricing	Brak Interpretation enum	ROZSZERZENIE
P3	hexabid-pricing	Brak ApplicabilityCondition	ROZSZERZENIE
P3	hexabid-payment-core	Brak AccountType / ChartOfAccounts / Ledger	ROZSZERZENIE
P3	hexabid-payment-core	Stary Price zamiast Money	ROZSZERZENIE
P3	hexabid-rules	Brak RuleVersion / persistencji	ROZSZERZENIE
P3	hexabid-rules-adapter	Niepodpięty do pricing / statements	ROZSZERZENIE
P3	hexabid-statements	Brak AuctionKind sealed hierarchy	ROZSZERZENIE
P3	hexabid-authorization-core	ResourceType tylko AUCTION/REPORT	ROZSZERZENIE
P3	hexabid-auth-core	Brak VerificationLevel	ROZSZERZENIE
P3	Nowy: hexabid-variance-core	Całościowa implementacja M10	NOWY MODUŁ
P4	Nowy: hexabid-graph-core	Wyodrębnienie z statements	NOWY MODUŁ

Plan fazowy

Faza 0: Bugfix [green] ☐ WYKONANE[/]

Cel: wyeliminować wszystkie P1 (błędne obliczenia, nieegzekwowane niezmienniki).

1. ☐ Naprawić `createExciseCalculator()` ExciseType (pricing) — switch na PERCENTAGE/PER_UNIT
2. ☐ Naprawić `isBalanced()` + dodać drugi entry w ProcessPaymentUseCase (payment-core)
3. ☐ Naprawić `Quantity.subtract()` (quantity)

Ryzyko: niskie. Zmiany lokalne, objęte istniejącymi testami.

Faza 1: Czystość heksagonalna [green] ☐ CZĘŚCIOWO[/]

Cel: usunąć naruszenia granic heksagonalnych – logika domenowa nie może być w adapterach, a framework nie może być w domenie.

1. ☐ Przenieść Spring `@Component/@EventListener` z payment-core do adaptera
2. — Przenieść wadium z REST adaptera do payment-core lub core
3. — Oznaczyć/usunąć `spring-boot-starter-security` z auth-core — deferowane (wymaga zmiany strategii autoryzacji)
4. — Rozdzielić `ProductFacade` na port + adapter — deferowane
5. ☐ Zabezpieczyć `InventoryEntry.addInstance()` przed duplikatami

Faza 2: Wzmocnienie modeli domenowych [green] ☐ CZĘŚCIOWO[/]

Cel: podnieść jakość istniejących modułów przez wzbogacenie modeli.

1. — Wzbogacić `Party` o zachowania (`hasRole()`, `isBuyer()`, `isSeller()`) — deferowane
2. ☐ Naprawić `Auction.edit()` — implementacja EditAuctionService
3. ☐ Dodać guard stanu w `assignParticipationPolicyTemplate()` (w ramach EditAuctionService)
4. ☐ Dodać zdarzenia domenowe w statements — `StatementDomainEvent` + `StatementEventPublisher`
5. — Wyeliminować duplikację kaskady w statements — deferowane (kosmetyka)
6. ☐ Dodać `Availability`, `Reservation` w inventory
7. — Dodać `Order` aggregate w core — deferowane

Faza 3: Zaawansowane inspiracje archetypowe [yellow]— DEFEROWANE[/]

Cel: implementacja konceptów z kursu AO podnoszących jakość i edukacyjność.

Większość pozycji oznaczona do oceny biznesowej. Nie realizowano w ramach aktualnej kampanii modernizacyjnej.

1. Dodać **Validity**, **Interpretation**, **ApplicabilityCondition** w pricing
2. Dodać **Variant** first-class w product
3. Dodać **PricingCatalog** port
4. Dodać **Variance** core module
5. Dodać **RuleVersion** + persistencję w rules
6. Podpiąć **RuleAlgebra** do pricing applicability i statements phase gating
7. Rozszerzyć **ResourceType** w authorization-core
8. Dodać **VerificationLevel** w auth-core

Faza 4: Wyodrębnienia i refaktoringi [green]□ CZĘŚCIOWO[/]

Cel: maksymalizacja reużywalności i czystości architektonicznej.

1. — Wyodrębnić **hexabid-graph-core** z **hexabid-statements** — deferowane
2. — Dokończyć migrację **Price** □ **Money** we wszystkich modułach — deferowane
3. □ Dodać **AccountType**, **ChartOfAccounts**, **Ledger** w payment-core
4. — Dodać resource-level authorization w AuthorizationContext — deferowane

Podsumowanie: Co sprawia, że Hexabid jest (może być) wzorcowy

Cecha	Opis
Czystość heksagonalna	Moduły domenowe nie zależą od Spring/JPA (po 1 wyjątku: auth-core używa spring-boot-starter-security – deferowane). Porty i adaptery są wyraźnie rozdzielone. <code>hexabid-payment-core</code> ma usunięty <code>spring-context</code> .
Bogate modele domenowe	Auction (state machine z 8 statusami), Pricing (Calculator/Component), Rules (AST + 3 algebry), Statements (DAG) – każdy to textbookowy przykład.
Architektura modularna	30+ modułów Mavena z jawnymi zależnościami. DAG zależności jest acykliczny (quantity → product → inventory → core; pricing → core; rules → core).
Kontrakt-first API	OpenAPI YAML → wygenerowany kod (Java + TypeScript). Ręczne implementacje tylko w adapterach.
Testy na każdym poziomie	Unit (JUnit 5 + AssertJ), architektura (ArchUnit), integracja (REST Assured), E2E (Playwright). Każdy poziom testuje inną warstwę.
Archetypy jako inspiracja	Nie "wdrożyliśmy archetyp", ale "zobaczyliśmy problem biznesowy i rozwiązaliśmy go w sposób, który pokrywa się z archetypem". To znak dojrzałego modelowania.

Kluczowe wskaźniki jakości po modernizacji

- **5 bugów domenowych (P1)** → 2 – 3 naprawione (Quantity.subtract, isBalanced, ExciseType switch); 1 częściowo (AuctionPricing excise base – nadal otwarte); 1 nie był P1 blokerem
- **4 naruszenia heksagonalne (P2)** → 1 – payment-core Spring usunięty (3 naruszenia naprawione); auth-core spring-boot-starter-security pozostał do oceny
- **9 problemów projektowych (P2 DESIGN)** → ~5 – EditAuctionService, domain events, Availability/Reservation zrealizowane; Party, ProductFacade, Order deferowane
- **15 rozszerzeń (P3)** → **ocena biznesowa** – które faktycznie podnoszą wartość (większość deferowana)
- **Każdy moduł domenowy ma co najmniej jeden walor edukacyjny** udokumentowany